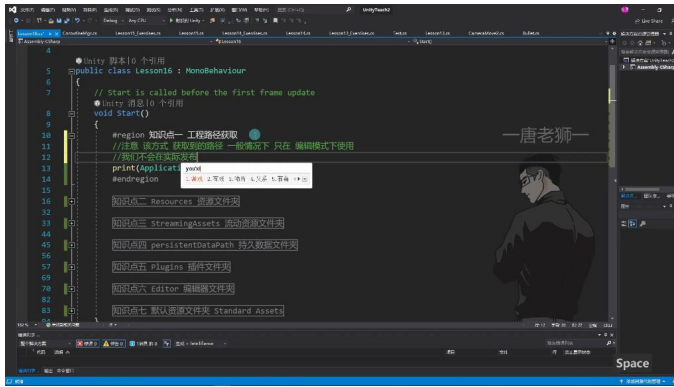
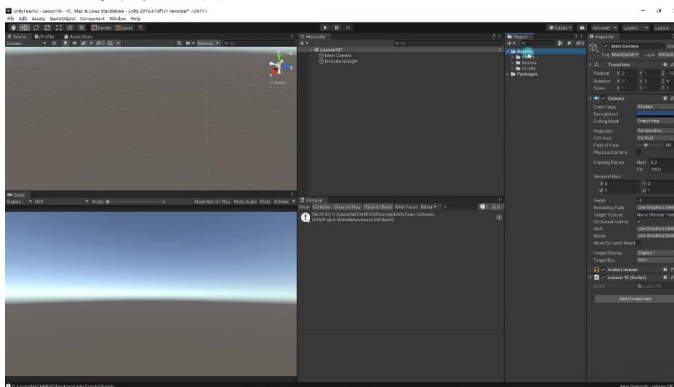


### 3) 注意事项



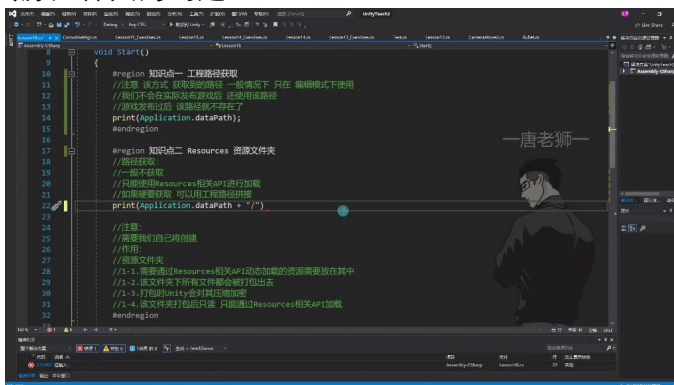
- 
- 使用限制:
  - 仅在编辑模式下有效
  - 发布后路径不可用（会被打包加密）
  - 主要用于开发阶段的测试用途
- 路径特点:
  - 开发阶段对应实际资源文件夹
  - 发布后资源会被压缩到包内
  - 无法通过常规文件操作访问

## 2. Resources资源文件夹 03:04



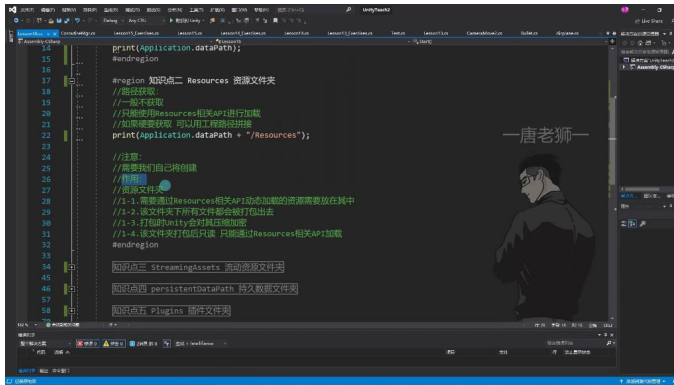
- 
- 创建要求: 必须手动在Assets文件夹下创建, 文件夹名称必须严格拼写为"Resources" (复数形式), 否则无法通过API加载资源
- 命名注意: 拼写错误将导致后续无法通过API加载资源, 创建时需特别注意复数形式和大小写

### 1) 资源文件夹的创建 03:12



- 
- 路径获取方式: 可通过工程路径拼接Application.dataPath + "/Resources"获取
- 路径使用限制: 仅在编辑模式下有效, 发布后该路径不存在, 实际开发中不建议使用路径获取方式

### 2) 资源文件夹的路径获取 03:53

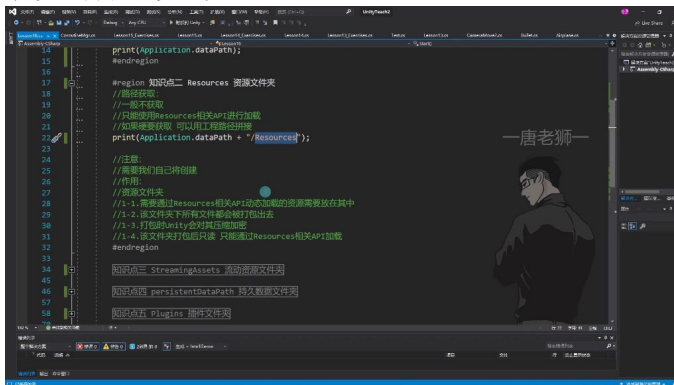


- 加载限制：只能使用Resources.Load等专用API进行资源加载
- 路径拼接示例：print(Application.dataPath + "/Resources")可打印完整路径（仅用于调试）

### 3) 资源文件夹的打包与加密 05:37

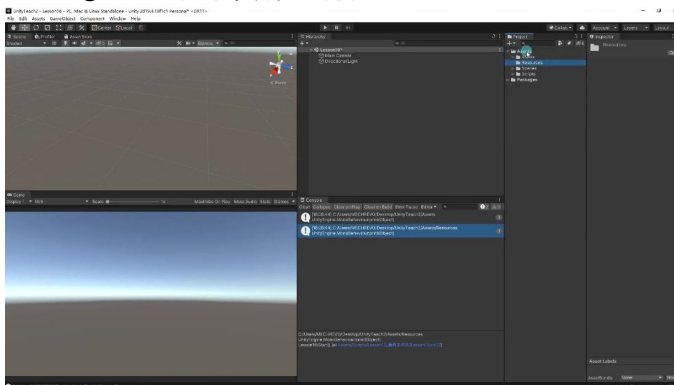
- 打包特性：
  - 所有文件都会被打包发布
  - Unity会进行压缩加密处理
  - 打包后变为只读状态
- 使用建议：不要存放所有资源，只放置需要动态加载的关键资源，避免包体过大

### 4) 资源文件夹的加载方式 06:05



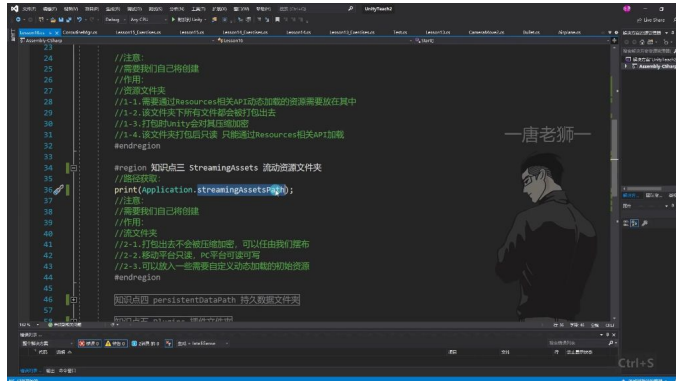
- 核心功能：专门用于存放需要通过Resources API动态加载的资源
- 最佳实践：
  - 建议新建专门的美术资源文件夹进行分类管理
  - 避免将非必要资源放入其中导致包体膨胀
  - 实际开发中主要使用Resources.Load等API进行资源加载而非路径访问

### 3. StreamingAssets流动资源文件夹 07:00



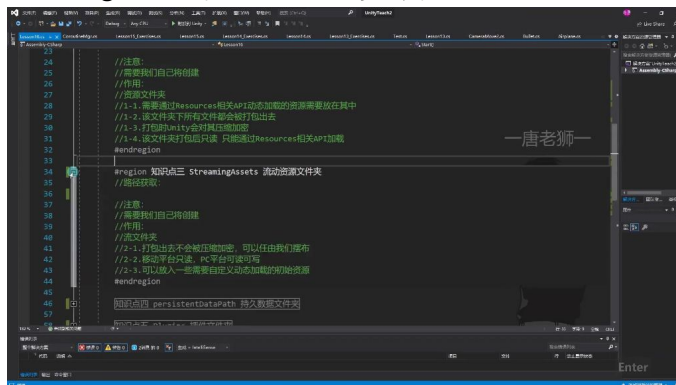
- 手动创建：需要手动在Assets目录下创建名为"StreamingAssets"的文件夹
- 命名规范：必须严格使用"StreamingAssets"作为文件夹名称，区分大小写

## 1) StreamingAssets路径获取 07:23



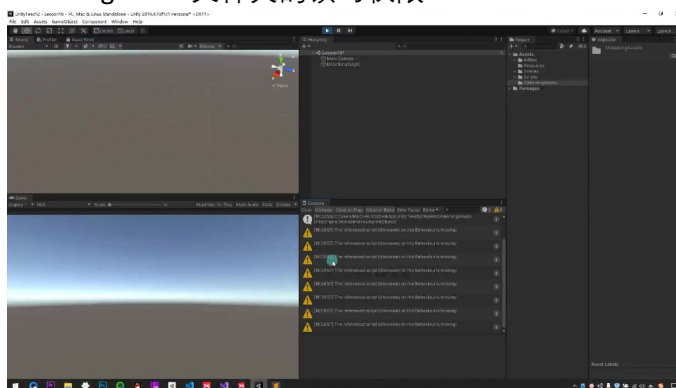
- 正确方法: 使用Application.streamingAssetsPath获取路径
- 错误示范: 不能使用Application.dataPath + "/StreamingAssets"拼接路径
- 平台差异: 在不同平台下打包后路径会变化, 拼接方式会导致路径错误
- 打印测试: 运行时打印结果为Assets/StreamingAssets

## 2) StreamingAssets与Resources的区别 08:46



- 压缩加密:
  - Resources: 打包时会被Unity压缩加密
  - StreamingAssets: 保持原始状态不压缩加密
- 加载方式:
  - Resources: 必须使用Resources API加载
  - StreamingAssets: 可使用C#原生文件操作API加载
- 可见性:
  - Resources: 所有内容强制打包
  - StreamingAssets: 内容保持原始文件形式

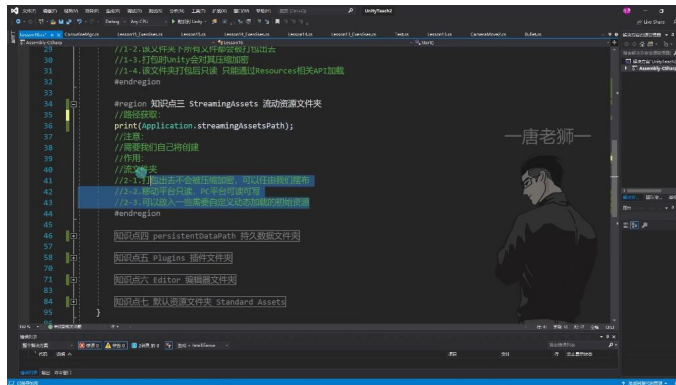
## 3) StreamingAssets文件夹的读写权限 10:28



- 移动平台: iOS/Android平台下为只读
- PC平台: Windows/Mac/Linux平台可读可写

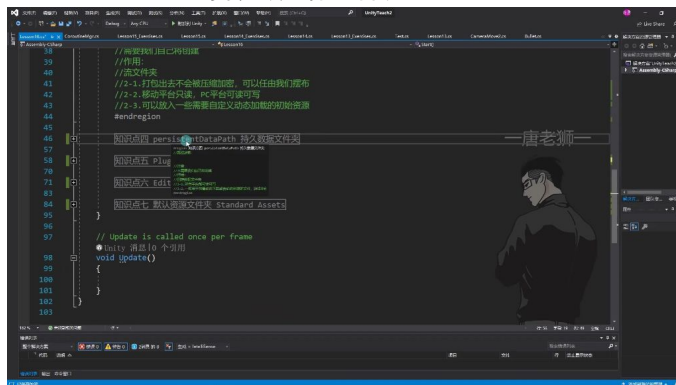
- **编辑模式**：在Unity编辑器内显示为可读写
- **重要限制**：打包后Application.dataPath路径将失效

#### 4) StreamingAssets文件夹的用途 10:47



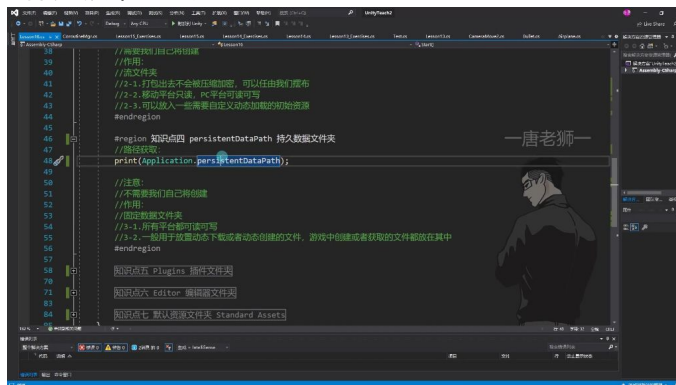
- **初始资源**：存放需要自定义加载的初始资源
- **配置文件**：适合放置XML/JSON等配置文件
- **热更新准备**：为资源热更新提供基础支持
- **特殊格式**：存放Unity不直接支持的特定格式文件
- **开发建议**：适合存放不需要Unity处理的原始资源文件

#### 4. persistentDataPath持久数据文件夹 11:24



- **文件夹特性 11:26**
- **自动创建**：不需要手动创建该文件夹，Unity会自动生成
- **平台差异性**：在不同平台下路径不同，无法在Unity编辑器中直接查看
- **存储位置**：存储在设备本地（如PC的C盘用户目录或手机存储中），与用户名相关

#### 2) 文件夹获取方式 11:52



- **代码获取**：通过Application.persistentDataPath获取路径
- **打印示例**：print(Application.persistentDataPath);

#### 3) 文件夹路径差异 12:01

- **PC平台示例**：C:\Users\[用户名]\AppData\LocalLow\DefaultCompany\UnityTeach2



- **移动平台**：路径会随设备不同而变化
- **不可见性**：路径不在项目Assets文件夹下，而是存储在设备本地

#### 4) 文件夹作用 12:32

- **主要用途**：存储持久化数据
- **具体功能**：
  - 保存玩家游戏数据
  - 热更新资源存储
  - 动态下载内容存储

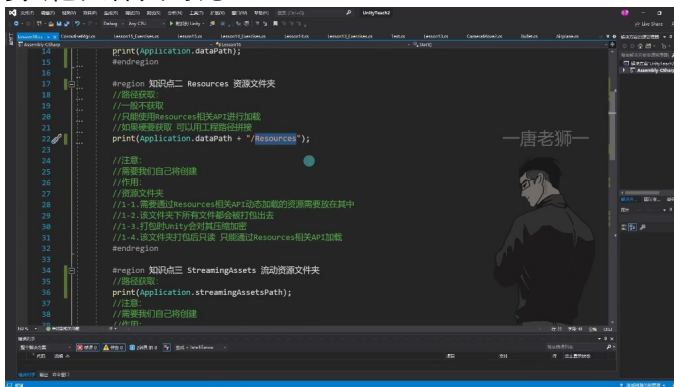
#### 5) 文件夹读写权限 12:52

- **全平台支持**：所有平台都可读可写
- **与StreamingAssets对比**：StreamingAssets是只读的，而persistentDataPath可读写

#### 6) 文件夹应用场景 13:04

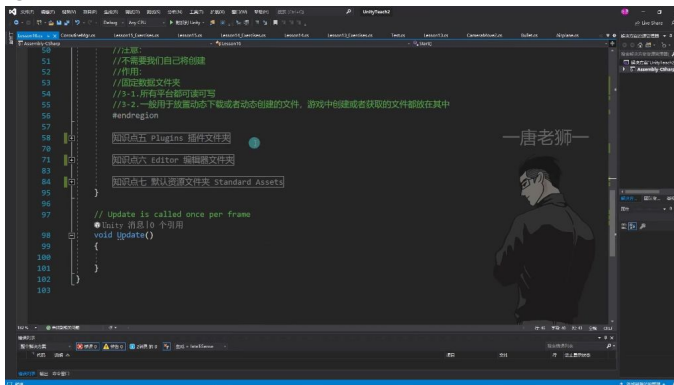
- **动态文件存储**：适合放置动态下载或动态创建的文件
- **游戏数据存储**：游戏中创建或获取的文件都应放在其中
- **典型用例**：
  - 玩家存档数据
  - 下载的游戏资源
  - 运行时生成的文件

#### 7) 与其他文件夹对比 13:54



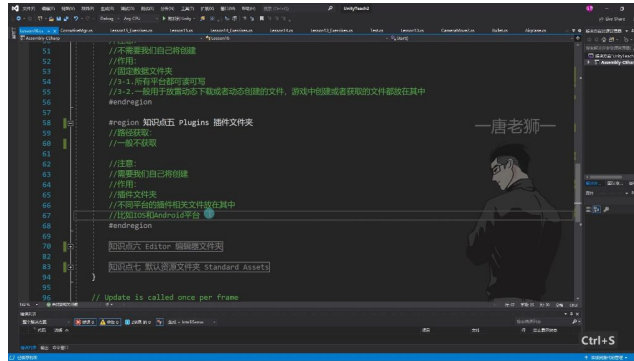
- **与Resources对比**：
  - Resources只读且加密，persistentDataPath可读写
  - Resources需要通过特定API加载，persistentDataPath可直接操作
- **与StreamingAssets对比**：
  - StreamingAssets只读，persistentDataPath可读写
  - StreamingAssets移动平台只读，PC平台可读可写
  - persistentDataPath在所有平台都可读写

#### 5. Plugins插件文件夹 14:41

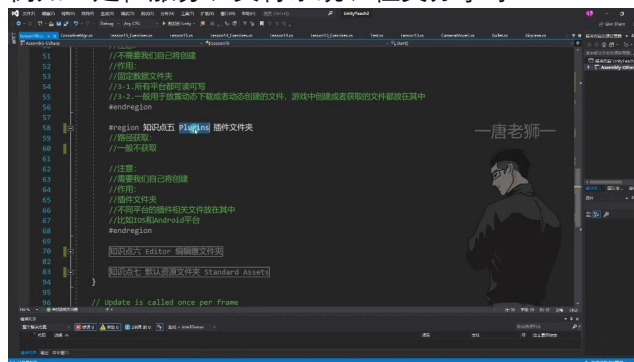


- **创建方式**：需要手动在Assets文件夹下创建，路径为Assets/Plugins

- **使用频率：** 在开发初期和基础学习阶段很少会用到这个文件夹
- **主要作用：** 存放不同平台的插件相关文件
- **典型应用：**
  - iOS和Android平台的特定功能插件
  - 第三方SDK集成文件
  - 原生平台开发的功能包

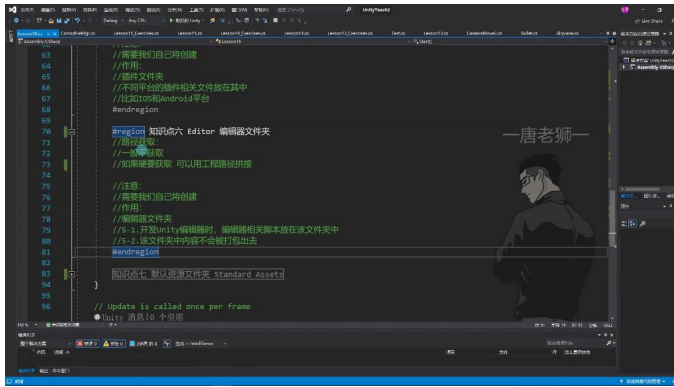


- **平台适配：**
  - 用于存放针对不同平台的特定实现
  - 例如iOS的定位功能、Android的支付SDK等
- **开发流程：**
  - 原生平台开发功能包
  - 以插件形式集成到Unity中
  - 通过C#代码调用插件功能
- **实际应用场景：**
  - 企业项目中常见，用于集成第三方服务
  - 需要调用平台特有功能时使用
  - 例如：定位服务、支付系统、社交分享等



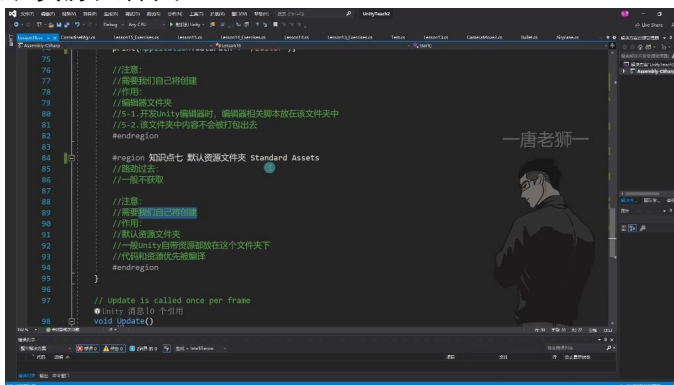
- **学习建议：**
  - 现阶段只需了解其基本概念
  - 实际应用会在SDK接入专题中详细讲解
- **特殊说明：**
  - 文件夹需要手动创建
  - 一般不直接获取或操作该文件夹路径
  - 主要用于存放跨平台开发的插件文件

## 6. Editor编辑器文件夹 16:30



- 
- **创建方式:** 需要手动在Assets文件夹下创建, 命名为"Editor"
- **路径获取:**
  - 一般不主动获取路径
  - 特殊需求时可使用工程路径拼接: `print(Application.dataPath + "/Editor")`
- **核心作用:**
  - 存放Unity编辑器开发相关的脚本文件
  - 典型应用场景: 开发自定义Inspector窗口、编辑器菜单扩展等
- **打包特性:**
  - 该文件夹内容不会被打包到最终产品中
  - 仅用于编辑器环境开发
- **注意事项:**
  - 必须手动创建, Unity不会自动生成
  - 编辑器功能代码必须放在此文件夹才能正常工作

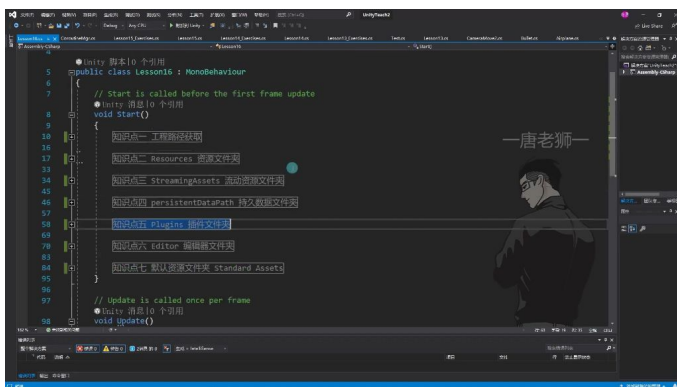
## 7. 默认资源文件夹 18:11



- 
- **创建方式:** 需要手动创建, 命名为"Standard Assets"
- **典型内容:**
  - Unity官方提供的默认资源
  - 从Asset Store下载的标准资源包
- **编译特性:**
  - 该文件夹下的代码和资源会优先编译
  - 资源加载具有更高优先级
- **使用现状:**
  - 商业项目中较少使用
  - 主要用于学习或快速原型开发
- **注意事项:**
  - 与Resources文件夹不同, 采用不同的加载机制
  - 官方资源导入时会自动创建此文件夹

## 8. 内容总结 19:25





- 
- **核心知识点：**
  - Resources：动态加载资源（只读）
  - StreamingAssets：存放默认资源（只读）
  - PersistentDataPath：可读写持久化数据存储
- **辅助知识点：**
  - Plugins：跨平台插件存放
  - Editor：编辑器扩展开发
  - StandardAssets：Unity默认资源
- **重点区分：**
  - 只读文件夹：Resources、StreamingAssets
  - 可读写文件夹：PersistentDataPath
  - 开发专用文件夹：Editor、Plugins
- **记忆要点：**
  - 必须手动创建的文件夹：Resources、Editor、StandardAssets
  - 自动生成的路径：PersistentDataPath
  - 打包排除的文件夹：Editor

## 二、知识小结

| 知识点                        | 核心内容                                                                                                                                            | 考试重点/易混淆点                                               | 难度系数 |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|------|
| 工程路径获取                     | 通过 Application.dataPath 获取工程路径（仅限编辑模式下有效，发布后路径失效）。                                                                                              | 注意：发布后路径不可用，仅用于开发测试。                                    | ★★   |
| Resources资源文件夹             | 手动创建 Resources 文件夹，用于动态加载资源（需通过 Resources API 加载）。 <ul style="list-style-type: none"> <li>- 打包时加密压缩，只读。</li> <li>- 所有文件默认打包，需谨慎放置资源。</li> </ul> | 易错点：文件夹名称必须为 Resources（复数形式）。<br>重点：仅存放需动态加载的资源，避免包体过大。 | ★★★  |
| StreamingAssets流<br>动资源文件夹 | 手动创建 StreamingAssets                                                                                                                            | 注意：路径需通过 Application.streamingAssetsPath                | ★★★  |

|                                      |                                                                                                                          |                                    |                  |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------|------------------|
|                                      | <p>文件夹，存放初始资源（如配置文件）。</p> <ul style="list-style-type: none"> <li>- 打包后不加密，可通过文件IO读取。</li> <li>- 移动端只读，PC端可读写。</li> </ul> | <p>ath 获取，不可拼接 dataPath。</p>       |                  |
| <b>PersistentDataPath</b><br>持久数据文件夹 | <p>自动生成路径（无需手动创建），用于存储运行时数据（如玩家存档、热更新文件）。</p> <ul style="list-style-type: none"> <li>- 全平台可读写。</li> </ul>                | <p>重点：唯一可写入的文件夹，适用于动态数据存储。</p>     | <p>★★★<br/>★</p> |
| <b>Plugins</b> 插件文件夹                 | <p>手动创建 Plugins 文件夹，存放平台相关插件（如 iOS/Android 原生库）。</p>                                                                     | <p>适用场景：SDK 接入或跨平台功能扩展。</p>        | <p>★★</p>        |
| <b>Editor</b> 编辑器文件夹                 | <p>手动创建 Editor 文件夹，存放编辑器拓展脚本。</p> <ul style="list-style-type: none"> <li>- 不打包到发布版本。</li> </ul>                          | <p>用途：Unity 编辑器功能开发（如自定义工具窗口）。</p> | <p>★★</p>        |
| <b>StandardAssets</b> 默认资源文件夹        | <p>Unity 官方资源包默认存放路径（现较少使用）。</p>                                                                                         | <p>注意：资源优先编译，商业项目罕见。</p>           | <p>★</p>         |